DNS

- 2 systems are there (in same network)

  A: 192.168.1.10

  B: 192.168.1.11 (hostname: db)

  network: 192.168.1.0

  - A can ping B with its IP

    ping 192.168.1.11 (it'll work)

  - But, A can't ping B with its name

    ping db (it'll not work)

- Name Resolution

  - Now, update **/etc/hosts** of A, about the server B

    192.168.1.11 db

    [A can ping B now using *ping db*]

  - The name (db) need not be same as B's hostname.

    192.168.1.11 some-server

    now, A can ping using *ping some-server* as well.

  - There can be multiple entries for a specific IP inside **/etc/hosts** .

  - *ping* might be disabled in any system, to check if he IP is getting resolved use ***nslookup*** command.

    nslookup <name>

    nslookup some-server

- DNS (Domain Name Server)

  - In previous method, we need to write the IP and Name mappings on each systems */etc/hosts* file.

    If any system's IP got changed, then need to update all the systems.

  - DNS server contains all the IP-Name mapping, and every system will ask DNS server to give the respective IP.

    Now, if any IP changes, only one place needs to be updated i.e. DNS Server.

  - Consider a DNS server with IP: *192.168.1.100* (same network as of A & B)

  - Then add this *nameserver* (DNS Server) in the file ***/etc/resolv.conf*** of each systems (A, B ..etc)

    nameserver 192.168.1.100

- Now every system will ask this nameserver for the IP of a specific name. And, in case of IP change, just update this nameserver.
- If any server that is not needed for all, and needed only for A.
  then, **/etc/hosts** of A can be updated with that new server so that only A can resolve its IP.
  192.168.1.115 test (A's /etc/hosts)
- Let say the same name (***test***) is being present inside nameserver as well but with another IP
  192.168.1.125 test [inside *nameserver* (192.168.1.100)]
- Now, if A try to ping this *test*,
  ping test
  first it'll check */etc/hosts* of A, then goes for nameserver.
  So, for A it'll be *192.168.1.115* where as for others (who doesn't contain entry for *test* in their respective /etc/hosts), it'll be *192.168.1.125* (nameserver's entry)
- This order can be changed.
  There is a file ***/etc/nsswitch.conf*** inside which one entry will be there
  hosts: files dns
  [files means /etc/hosts, dns means nameserver]
  It means first check */etc/hosts* and then check nameserver.
  If you write it opposite then it'll change
  hosts: dns files
- Lets say you are trying to ping a server which is not there in either list (/etc/hosts & nameserver),
  for example: *ping facebook.com*
  ~~It'll fail~~.
- In this case, you can add a nameserver who knows this server (facebook.com) inside */etc/resolv.conf* file.
  Now, /etc/resolv.conf has 2 nameservers configured:
  nameserver 192.168.1.100
  nameserver 8.8.8.8 (public DNS server provided by Google)
  [8.8.8.8 can extract all the domains throughout the internet; because it's a Recursive DNS service]

々 Domain name

- <sub-domain>.<sub-domain>.<domain>.<TLD>

  (sub-domain can be multiple, not limited to 2)

  - www.google.com, maps.google.com

    www, maps: sub-domain

    google: domain

    .com: TLD (Top Level Domain)

  - . is the Root -> then domain -> then subdomains.
- Full actual domain look like this,

  www.google.com.

  [that dot (.) at the end is Root]

- Flow of domain resolving to IP
  - Let say you tried to access *apps.google.com* .
  - it'll check your local ***/etc/hosts*** file.

    [it is not there let say]
  - It'll go to your organization's DNS server to check if there is any entry of this.

    [it is not there let say]
  - Either your */etc/resolv.conf* has a public DNS (nameserver 8.8.8.8) or your organization's DNS forward this to a public DNS.
  - Request goes to public DNS.
    - Public DNS are Recursive.

      It means they recursively fetch the IP from domain starting from root to domain level.
    - First, Recursive DNS will ask the Root DNS if any entry for *apps.google.com* is there.

      [Root DNS: .]

      Root DNS responds with the TLD (*.com*)'s DNS server.
    - Recursive DNS will ask TLD if there is entry of *apps.google.com* is there.

      [TLD: **.com** in our case]

      TLD responds with the *google.com* DNS server.
    - Recursive DNS will ask *google.com* DNS server about *apps.google.com*

      either *google.com* DNS server would have contained all the domains

like *maps.google.com*, *apps.google.com*, *www.google.com* ..etc.

Or there might be other DNS servers containing the sub-domains.

(in this case Recursive DNS ask that server about the domain *apps.google.com*)

- now the IP is fetched and it'll be stored in cache (in organization's DNS) for few seconds or minutes.

- Let say your organization has so many sub-domains like *hr, pay, drive, mail, sql* ..etc.

  And domain is: *mycompany.com*

  - Outsiders should give full domain to access these like *hr.mycompany.com*, *mail.mycompany.com* ..etc.
  - But people of the company itself, they shouldn't write full name. They should be able to access like *hr, pay, drive* ..etc.
  - For example, they just write *hr* and it'll be converted to *hr.mycompany.com*
  - But inside the internal DNS server, full domain is present like *hr.mycompany.com*, *mail.mycompany.com* ..etc.
  - You can write ***search*** in */etc/resolv.conf* like the below

    nameserver 192.168.1.100

    search mycompany.com prod.mycompany.com

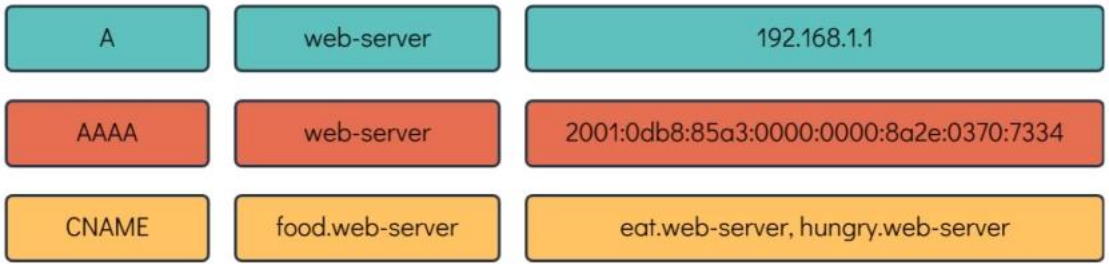
    - Order matters, it'll be checked from left to right till the valid domain is got.

      hr.mycompany.com, *hr.prod.mycompany.com* ...like this.

- Record Types
  - **A**

    name to IPv4
  - **AAAA**

    name to IPv6
  - **CNAME**

    name to name
  - 

- **ping** is not always the right tool to check DNS resolution.
  - **nslookup** can be used to query a hostname from a DNS Server.

    ~~It doesn't consider entries inside */etc/hosts* file contents~~.
  - **dig** returns more details than *nslookup*. And it is used widely.
- DNS is interface scoped.
  - Let say you have 2 network interfaces, one connected to internal DNS and another to Internet.
  - In modern systems, */etc/resolv.conf* doesn't contain the nameservers directly, instead it contains the below

    ```
    nameserver 127.0.0.53
    options edns0 trust-ad
    search .
    ```

    It means "Send all DNS queries to local **systemd-resolved** service"
  - **systemd-resolved** is a *systemctl service* [you can see via *systemctl cat systemd-resolved*] which internally maintains DNS ***per interface***.
  - In modern system, ~~*/etc/resolv.conf* is not changed mostly~~, rather DNS is provided at interface level.
  - **resolvectl status**

    this gives the DNS details of each interfaces.
  - If 2 interfaces contains 2 DNS servers, and in both one common name is present with different IPs.

    test: 192.168.5.16 (dns server 1)

test: 192.168.10.17 (dns server 2)

here the *systemd-resolved* is the main. Whatever it selects become the final.

- Global nameservers can also be added.

  /etc/systemd/resolved.conf

  [*space separated* dns server IPs. Like, *8.8.8.8 1.1.1.1*]

  - Question is: through which interface global DNS will be resolved?
    - It'll be fetched from the routing table.
    - If not present, then default gateway.

- Whatever the type of nameservers (interface level or global), **systemd-resolved** only choose the nameserver.

-

Linux Namespace

- Network Namespace is a separate network inside the host.
  - Host and Namespace doesn't know anything about each other.
- The namespace will have its own *network interfaces, arp table, routing table* (independent of Host).
- I've 2 vms: vm1, vm2

  vm1: 10.10.10.10/24

  vm2: 10.10.10.11/24
- vm1 and vm2 can ping each other

  because both belong to same LAN (10.10.10.x).

```
vagrant@vm1:~$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         10.0.2.2        0.0.0.0         UG    100    0        0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
10.0.2.2        0.0.0.0         255.255.255.255 UH    100    0        0 enp0s3
10.10.10.0      0.0.0.0         255.255.255.0   U     0      0        0 enp0s8
192.168.0.1     10.0.2.2        255.255.255.255 UGH   100    0        0 enp0s3
```

[vm1 route table]

  Gateway 0.0.0.0 means this subnet (*10.10.10.0*) Gateway/Router is not needed because both belong to same ***subnet***.

  10.10.10.0 is added on ***enpos8*** (means this interface is connected to private network i.e. vm to vm communication)

  route doesn't depend only on Gateway, it needs to know through which network interface traffic should be sent. However, ***kernel finds it by itself seeing the gateway IP.***
- **ip add**

  display all the network interfaces names like *lo* (loopback), *enpos3*, *enpos8* ..etc.

  en: ethernet; po: PCI bus 0; s3: slot 3

  enpos8: ethernet device on slot 8

  in our VM (vm1, vm2), *lo enpos3 enpos8* will be there.

  s3 is for vm-to-vm communication (inter LAN), ***s8*** is for internet access (NAT).
- To create a namespace

  ip netns add <namespace name>

- We'll create 2 namespaces having name *red*, *blue* .

```
root@vm1:~# ip netns add red
root@vm1:~# ip netns add blue
```

- **ip netns list**

  to list all the namespaces

```
root@vm1:~# ip netns list
blue
red
```

- To execute any command inside a namespace

  ip netns exec <network namespace name> <command to execute>

  - ip netns exec red ip add

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group de
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

    [only loopback is present]

    [same for ***blue*** namespace as well]

  - ip netns exec red route

```
root@vm1:~# ip netns exec red ip route
root@vm1:~#
```

    [no route is present]

    [same for ***blue*** namespace as well]

- Now we'll create a **veth pair** (just like a virtual cable) to connect the 2 namespaces.

  ip link add <veth's one end name> **type veth peer name** <veth's another end name>

  [when you create a **veth pair**, it creates **2** *veth* network interface in the host and connect them internally. From outside, it looks 2 ends of a virtual ethernet cable]

  - ip link add veth-red type veth peer name veth-blue

```
4: veth-blue@veth-red: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff
5: veth-red@veth-blue: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff
```

    [ip add]

    [2 veth network interface; internally connected; both represents one one end]

    veth-blue@veth-red -> veth-blue interface is connected to veth-red

    veth-red@veth-blue -> veth-red interface is connected to veth-blue

- If both the ends of a veth pair present in same place (ex: both in host), then it is displayed like

  end1-name@end2-name (end1 to end2)

  end2-name@end1-name (end2 to end1)

  if both are present in different namespaces (host & namespace; or namespace1 & namespace2), then it'll be displayed like

  end1-name@if<end2's index at its namespace>

  end2-name@if<end1's index at its namespace>

- To connect ***veth*** ends to namespaces

  ip link set <veth end name> **netns** <network namespace name>

- Connect *veth-red* end to the namespace ***red***

  ip link set veth-red netns red

  - Now if you see,

    ```
    4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
        link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
    ```

    link-netns red it means other end is present in namespace red.

  - *veth-red@veth-blue* is gone

    because *veth-red* is now connected to a namespace. Host linux (host) cannot show peer interface directly it moved to namespace.

  - *veth-blue@veth-red* got updated to *veth-blue@if5* because veth-red's index inside namespace **red** is ***5***.

    ```
    root@vm1:~# ip netns exec red ip add
    1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
        link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
        link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    ```

    [ip netns exec red ip add]

    And also, here, *veth-red@**if4*** because *veth-blue* is at 4th index .

    link-netnsid 0 means other end present in host.

- Connect *veth-blue* to the namespace ***blue***

  ip link set veth-blue netns blue

  - Now, *veth-blue* will be gone from host because it belongs to namespace blue.

- Network interfaces in both namespace now

  - Namespace **red**:

    ip netns exec red ip add

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue
```

- **link-netns blue** : other end is in namespace blue

- Namespace **blue**:

  ip netns exec blue ip add

```
root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
```

- **link-netns red** : other end is in namespace red

- Now make the network interfaces in both namespaces UP.

  ip netns exec <network namespace name> **ip link set** <interface name> **up**

  - ip netns exec red ip link set lo up

    ip netns exec red ip link set veth-red up

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
5: veth-red@if4: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue
```

    LOWERLAYERDOWN: means it is up, but its other end (veth-blue) is down.

  - ip netns exec red ip link set lo up

    ip netns exec red ip link set veth-red up

```
root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
    inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
       valid_lft forever preferred_lft forever
```

  - Now, both *veth-red* and *veth-blue* are UP.

- Assign IP to both the namespace

  ip netns exec <namespace name> **ip addr add** <IP>/<subnet mask length>

  dev <interface name>

- Assign in namespace **red**:

  ip netns exec red ip addr add *192.168.15.**2**/24* dev *veth-red*

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN gr
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
5: veth-red@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue
    inet 192.168.15.2/24 scope global veth-red
       valid_lft forever preferred_lft forever
    inet6 fe80::dc62:4dff:fe8e:dab9/64 scope link
       valid_lft forever preferred_lft forever
```

  [IP is assigned]

- Assign in namespace **blue**:

  ip netns exec blue ip addr add *192.168.15.**3**/24* dev *veth-blue*

```
root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN g
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc no
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
    inet 192.168.15.3/24 scope global veth-blue
       valid_lft forever preferred_lft forever
    inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
       valid_lft forever preferred_lft forever
```

- The IPs are 192.168.15.2/24 & 192.168.15.3/24,

  so private LAN's network IP will be: **198.168.15.0**

  & subnet mask will be: **255.255.255.0**

- Ping the namespaces from each other. It'll work fine.

  - From namespace **red**:

    ip netns exec red ping -c 4 192.168.15.3

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=0.041 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.135 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.060 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.060 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3071ms
rtt min/avg/max/mdev = 0.041/0.074/0.135/0.036 ms
```

- From namespace **blue**:

  ip netns exec blue ping -c 4 192.168.15.2

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.210 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.057/0.112/0.210/0.058 ms
```

- Check **arp** table in both namespace, it should be updated.
  - In namespace **red**:

```
root@vm1:~# ip netns exec red arp
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.15.3             ether   ba:86:30:88:3b:b8   C                     veth-red
```

  - In namespace **blue**:

```
root@vm1:~# ip netns exec blue arp
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.15.2             ether   de:62:4d:8e:da:b9   C                     veth-blue
```

- But in Host's arp table, nothing related to namespaces will be there.

  Because both are separate.

```
root@vm1:~# arp
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.2.2                 ether   52:54:00:12:35:00   C                     enp0s3
10.10.10.11              ether   08:00:27:ad:99:10   C                     enp0s8
```

- Try to ping the namespaces from Host.

  ping -c 4 <namespace network interface IP>

  - Ping to namespace **red**:

    ping -c 4 192.168.15.2

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[4]+  Stopped                 ping -c 4 192.168.15.2
```

    ~~FAILED~~

  - Ping to namespace red

    ping -c 4 192.168.15.3

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
^Z
[5]+  Stopped                 ping -c 4 192.168.15.3
```

    ~~FAILED~~

- It is failing because there is no connection between host and namespaces.

  Create a veth pair and connect that to host and namespace, then only you can ping.

  [however, both end present in the host initially, just connect one to a namespace]

- To test connection between host and a namespace (let **red**), create a veth pair and connect one end to the namespace.

  Another end will be by default connected to host.

  - ip link add *veth2-red* type veth peer name *veth2-red-host* [create veth pair]
  - ip link set *veth2-red* netns *red* [connect veth2-red end to namespace red]
  - ip netns exec *red* ip link set *veth2-red up* [make the link up in namespace]
  - ip link set *veth2-red-host* up [make the link up in host]
  - netns exec red ip addr add *192.168.**25**.2/24* **dev** *veth2-red* [assign ip in namespace]
  - ip addr add *192.168.25.3/24* dev *veth2-red-host* [assign ip in host]
  - ping -c 4 192.168.**25**.2 (ping from host to namespace)

```
root@vm1:~# ping -c 4 192.168.25.2
PING 192.168.25.2 (192.168.25.2) 56(84) bytes of data.
64 bytes from 192.168.25.2: icmp_seq=1 ttl=64 time=2.54 ms
64 bytes from 192.168.25.2: icmp_seq=2 ttl=64 time=0.058 ms
64 bytes from 192.168.25.2: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 192.168.25.2: icmp_seq=4 ttl=64 time=0.086 ms

--- 192.168.25.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3041ms
rtt min/avg/max/mdev = 0.058/0.688/2.542/1.070 ms
```

  - ip netns exec red ping -c 4 192.168.**25**.3 (ping from namespace to host)

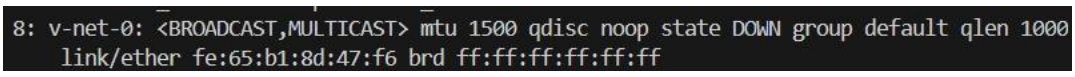
```
root@vm1:~# ip netns exec red ping -c 4 192.168.25.3
PING 192.168.25.3 (192.168.25.3) 56(84) bytes of data.
64 bytes from 192.168.25.3: icmp_seq=1 ttl=64 time=0.327 ms
64 bytes from 192.168.25.3: icmp_seq=2 ttl=64 time=0.062 ms
64 bytes from 192.168.25.3: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from 192.168.25.3: icmp_seq=4 ttl=64 time=0.307 ms

--- 192.168.25.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.052/0.187/0.327/0.130 ms
```

- Here we did the same thing, before connected *veth pair* from namespace to namespace; not connected *veth pair* from host to namespace.
  But, we had pinged to *192.168.**25**.2* (not **15.2** which is previous veth's subnet)
- You can add gateway in host
  ip route add <network-ip> **via** <gateway-ip>
  ip route add 192.168.15.0 via 192.168.25.2
  but need to enable packet forwarding inside namespace **red**. Because linux doesn't forward packets by default.
- But, one issue will be there,
  we configured 192.168.15.0 via 192.168.25.2
  but the namespace **blue** also have this subnet, for that also it'll send to **red** namespace because gateway is *192.168.25.2*
- We'll do this in broader picture; not namespace level;

- We had 2 namespaces so one veth and directly connect both.
  In case of multiple namespaces, need to create a bridge.
  [inside linux, **virtual switch** can be created]

- Delete all the veth pairs.
  ip link delete <any one end name>
  [it'll delete the other name by default]
  - ip netns exec red ip link delete veth-red
  - ip link delete veth2-red-host

- Create a bridge
  ip link add <bridge interface name> **type bridge**
  - ip link add v-net-0 type bridge
  - 

    [ip add]
    its coming as a normal network interface only.
  - Linux bridge is nothing but a network interface like others.
  - The bridge belongs to the namespace where it is created.
    If created in host, it belong to the host.
    If created in a namespace, it belong to that namespace.

- Make the bridge network interface UP.

  ip link set *v-net-0* up

- Now create a ***veth pair***, connect one end to the namespace **red** and another to **bridge**.
  - ip link add *veth-red* type veth peer name *veth-red-br* [create veth pair]
  - ip link set *veth-red* netns *red* [connect one end to namespace *red*]
  - Now in host [ip add]

    ```
    8: v-net-0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
        link/ether fe:65:b1:8d:47:f6 brd ff:ff:ff:ff:ff:ff
    9: veth-red-br@if10: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
        link/ether be:57:96:43:87:7c brd ff:ff:ff:ff:ff:ff link-netns red
    ```

  - ip link set *veth-red-br* master *v-net-0* [connect another end to bridge]

    [for master node's interface, use **master** <host's interface name>]
  - ip netns exec *red* ip link set *veth-red up* [make namespace end up]
  - ip link set *veth-red-br* up [make bridge end up]

- Do the same for ***blue*** namespace.
  - ip link add *veth-blue* type veth peer name *veth-blue-br* [create veth pair]
  - ip link set *veth-blue* netns *blue* [connect one end to namespace *red*]
  - Now in host [ip add]

    ```
    11: veth-blue-br@if12: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
        link/ether 06:8d:2b:07:c6:00 brd ff:ff:ff:ff:ff:ff link-netns blue
    ```

  - ip link set *veth-blue-br* master *v-net-0* [connect another end to bridge]
  - ip netns exec *blue* ip link set *veth-blue up* [make namespace end up]
  - ip link set *veth-blue-br* up [make bridge end up]

- Now assign IPs to both red and blue namespace
  - ip netns exec *red* ip addr add *192.168.15.2/**24*** dev *veth-red* [assign IP to red namespace]
  - ip netns exec *blue* ip addr add *192.168.15.3/**24*** dev *veth-blue* [assign IP to blue namespace]

- Now, ping from one namespace to another will be successful.

- Ping red to blue

  ip netns exec red ping -c 4 192.168.15.3

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=3.58 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.103 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.080 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3044ms
rtt min/avg/max/mdev = 0.080/0.977/3.580/1.502 ms
```

- Ping from blue to red

  ip netns exec red ping -c 4 192.168.15.3

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.501 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.107 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.068 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 0.060/0.184/0.501/0.183 ms
```

- But, Host cannot ping either of these namespace
  - Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[10]+  Stopped                 ping -c 4 192.168.15.2
```

  - Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data
^Z
[11]+  Stopped                 ping -c 4 192.168.15.3
```

- Add IP address to the bridge network interface (*v-net-0*)

  ip addr add *192.168.15.5/24* dev *v-net-0*

  - Because till now host doesn't know any subnet of 102.168.15.x

- Now try to ping again.
  - Not it'll be successful

    because, host has now a interface having the same subnet as that of the namespaces.

- Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=3.45 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.373 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.111 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.057/0.998/3.452/1.421 ms
```

- Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=2.51 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.248 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.117 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.090 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.090/0.740/2.506/1.021 ms
```

- Ping from namespace to host should also work.

  But to **102.168.15.5** (bridge network interface IP) only.

  It'll not work for *10.10.10.10* (host (vm1) IP)

  - Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=0.894 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.059 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3043ms
rtt min/avg/max/mdev = 0.059/0.275/0.894/0.357 ms
```

  - Namespace blue to host.

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.066 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 0.066/0.448/1.581/0.654 ms
```

  - ~~To 10.10.10.10 will not work~~

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

['Network is unreachable' comes when **gateway is not there**]

- Add gateway in the namespaces

  network subnet: 10.10.10.0

  gateway: 192.168.15.5 [bridge network interface's IP]

  [means to communicate with *10.10.10.0*, route to *192.168.15.5* (bridge's IP)]

  - ip netns exec red ip route add *10.10.10.0/**24*** via *192.168.15.5* [in namespace red]
  - ip netns exec blue ip route add *10.10.10.0/**24*** via *192.168.15.5* [in namespace blue]

    [always add the subnet **/24**, otherwise it'll take **/32** by default]

- Now try to ping host (10.10.10.10) from namespaces
  - Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=3.43 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.074 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.085 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.122 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3037ms
rtt min/avg/max/mdev = 0.074/0.928/3.431/1.445 ms
```

  - Namespace blue to host

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=0.917 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.097 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.063 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.075 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.063/0.288/0.917/0.363 ms
```

- Now try to ping **vm2** (10.10.10.**11**) from the namespaces.

  It'll ping (because gateway is there) but response will not come.

  - Namespace red to vm2

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[14]+  Stopped                 ip netns exec red ping -c 4 10.10.10.11
```

- Namespace blue to vm2

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[15]+  Stopped                 ip netns exec blue ping -c 4 10.10.10.11
```

- It is because, packet is going to vm2 (10.10.10.11), but it doesn't know about **192.168.15.x** because it belongs to private network of **vm1** (host).
- Either add a gateway in **vm2** or enable **NAT** in vm1.

- Option-1: adding gateway in **vm2**

  network: 192.168.15.0

  gateway: 10.10.10.10 (vm1 / host)

  [means to communicate with *192.168.15.0* subnet, route to *10.10.10.10*]

  - ip route add 192.168.15.0/24 via 10.10.10.10 [vm2]
  -

```
root@vm2:~# ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.10.10.0/24 dev enp0s8 proto kernel scope link src 10.10.10.11
192.168.0.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
192.168.15.0/24 via 10.10.10.10 dev enp0s8
```

    [vm2]

    [ip route]

  - Even now ping from namespace to vm2 or vice-versa will not work.

    Because, IP forwarding is disable in vm1 (host)

    cat /proc/sys/net/ipv4/ip_forward

    0: disabled

    1: enabled

  - **vm1 OS** is acting as the **gateway** and **router** for different networks.

    10.10.10.x network & *192.168.15.x* network.

  - A router's main job is to receive packet from a network, and forward that to another network.

    Here, **vm1 OS** is acting as a router, receiving from one network and forwarding to another. So, packet forwarding needs to be enabled.

  - Enable ip forwarding

    echo 1 > /proc/sys/net/ipv4/ip_forward

- Now ping will work

```
root@vm2:~# ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=63 time=4.68 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=63 time=1.55 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=63 time=1.31 ms
^Z
[8]+  Stopped                 ping 192.168.15.2
```

[vm2 to namespace]

```
root@vm1:~# ip netns exec red ping 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=0.902 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=2.23 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.20 ms
^Z
[20]+  Stopped                 ip netns exec red ping 10.10.10.11
```

[namespace t vm2]

- In this case, **vm2** can directly communicate to **namespace** (of vm1).

- Option-2: using NAT
  - Previous option is not preferable because if multiple systems will be there, in each system we need to set route gateway.
  - If we enable NAT in **vm1**, it'll update the source IP before the packet exits.
    - Packet before NAT:

      src IP: 192.168.15.2 (red namespace)

      dest IP: 10.10.10.11 (vm2)
    - Packet after NAT:

      src IP: 10.10.10.10 (host/vm1 IP)

      dest IP: 10.10.10.11 (vm2) --> it doesn't change.
  - Now the packet can go to **vm2** easily, and vm2 as well send back the response.

    src IP: 10.10.10.11

    dest IP: 10.10.10.10 (because this was the source IP received at vm2)
  - vm1 (host) will receive the packet. And from the NAT table it knows that this packet belong to *192.168.15.2* (red namespace).

    Now it updates the packet's dest IP

    src IP: 10.10.10.11 (vm2, doesn't change)

    dest IP: 192.158.15.2 (red namespace)

- To setup this, First delete the created configs (vm2 route gateway).

  Don't disable IP packet forwarding (/proc/sys/net/ipv4/ip_forward) because NAT requires this.

- **iptables -t nat -A POSTROUTING -o <network interface name> -j MASQUERADE**

  -t: use NAT table

  POSTROUTING: modify packet before leaving

  <network interface name>: network interface connected to vm2

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu
    link/ether 08:00:27:c3:54:e8 brd ff:ff:ff:ff:
    inet 10.10.10.10/24 brd 10.10.10.255 scope gl
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fec3:54e8/64 scope link
       valid_lft forever preferred_lft forever
```

  [in my case **enpos8**]

  MASQUERADE: replace source IP with host/router IP

  [for incoming traffic, destination ip will be updated (host IP to namespace's IP). it looks like DNAT but it is not. Kernel automatically reverse NAT mapping because it remembers previously source IP was changed]

  - iptables -t nat -A POSTROUTING -o enpos8 -j MASQUERADE

- Now, from namespace to vm2 can be easily accessible.

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.74 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.83 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.03 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 1.027/2.123/3.736/0.992 ms
```

[red to vm2]

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.14 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.29 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.49 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=0.906 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.906/1.707/3.140/0.853 ms
```

[blue to vm2]

- But, **vm2** to **namespace** will not be accessible.

  Namespace to vm2:

  when vm2 receives the packet, *src ip: vm1's ip*, *dest ip: vm2's ip*

so, vm2 also sends the packet, *src ip: vm2's ip, dest ip: vm1's ip*

at the end, vm1 forward the packet to namespace.

[in this scenario, vm2 is sending packet to vm1 only, not to namespace]

- But, directly if you try to access namespace from vm2 it'll not work because vm2 doesn't know about namespace.

  If you want this only, then **route** needs to be added in **vm2** .

- **Port forwarding** can be used in this case.

  Vm2 will send packet to vm1 at a particular port. Vm1 will forward the packet to namespace.

  It is **DNAT**.

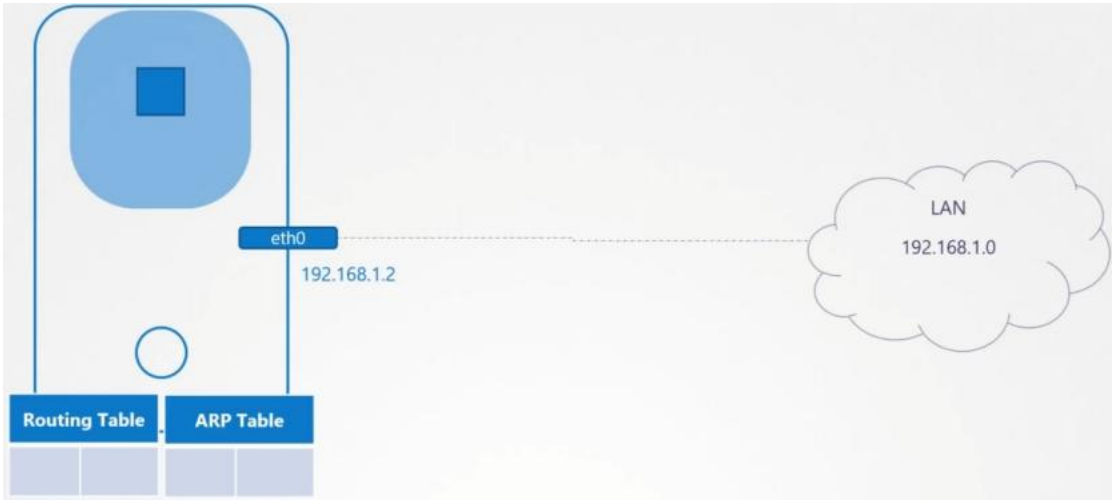
- **iptables -t nat -A PREROUTING -p tcp --dport** 8080 **-j DNAT --to-destination** 192.168.15.2:80

  Packets arriving on port 8080 should be redirected to namespace 192.168.15.2:80

- I ran a simple nodejs in namespace *red* on port 80,

  curl http://10.10.10.10:8080 [from vm2]

- **Creating namespace and checking the things** (extra part; no need)



[note the IP: ***192.168.1.2***]

the host might have connected to a LAN and got a IP from that swtich.

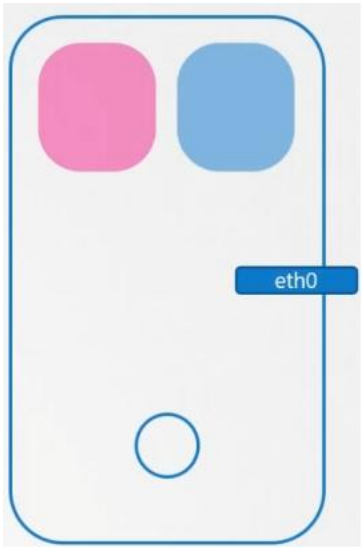
But, the namespace that will be created inside the Host will not have any idea about it.

It'll be completely different and will not be aware about the host.

- Lets say we created 2 network namespaced (**red** and **blue**) inside the host.

ip netns add red

ip netns add blue



- When you execute **ip link** on the host, it'll display some interfaces like

```
ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc state UN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qd
    link/ether 02:42:ac:11:00:08 brd ff:ff:ff:ff:ff:ff
```

But when you execute the same inside the namespaces

ip netns exec red ip link

only one loopback will be there

```
ip netns exec red   ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc
    link/loopback 00:00:00:00:00:00 brd 00:00
```

(similarly for blue namespace)

NOTE: whatever command needs to be executed inside the namespace,

ip netns exec <namespace name> <command to execute>

- Same for **arp** as well.

In the host:

```
arp
Address                  HWtype  HWaddress           Flags Mask
172.17.0.21              ether   02:42:ac:11:00:15   C
172.16.0.8               ether   06:fe:d3:b5:59:65   C
_gateway                 ether   02:42:d5:7a:84:8e   C
host01                   ether   02:42:ac:11:00:1c   C
```

```
ip netns exec red arp
Address                  HWtype  HWad
```

[nothing is there inside the namespace]

- Same for **route** as well.

In the host:

```
route
Kernel IP routing table
Destination     Gateway         Genmask         Flags
default         _gateway        0.0.0.0         UG
172.17.0.0      0.0.0.0         255.255.0.0     U
172.17.0.0      0.0.0.0         255.255.255.0   U
```

```
ip netns exec red route
Kernel IP routing table
Destination     Gateway         Genmask
```

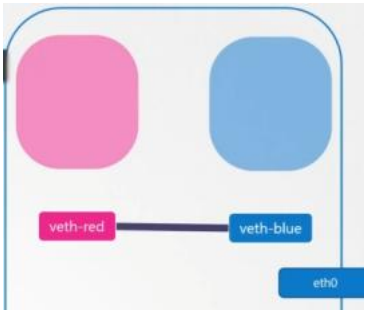
[nothing is there inside the namespace]

- To connect the 2 namespaces, we need to connect a cable between them. That cable is nothing but **veth pair**.
- Create a *veth pair*.

  ip link add veth-red type veth peer name veth-blue

  ```
  ip link add veth-red type veth peer name veth-blue
  ```

  here, *veth-red* and *veth-blue* are just the names of the 2 ends of the veth pair.
- When you create a veth, it is created in the HOST's namespace. You need to direct that towards the required namespace.

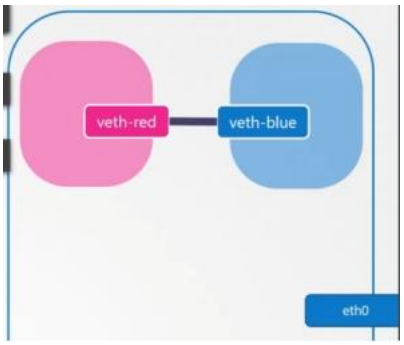
- **ip link set** <veth's end name> **netns** <network namespace name>

  ip link set veth-red netns red

  ip link set veth-blue netns blue

  ```
  ip link set veth-red netns red
  ```

  ```
  ip link set veth-blue netns blue
  ```

- Now we'll assign IP to each namespaces.

  ip -n <namespace name> **addr add** <ip address> **dev** <interface name like etho>

  (or)

  ip netns exec <namespace name> **ip addr add** <ip address> **dev** <interface name>

[both are same, **ip** command contains a flag **-n** to provide namespace so it can be used like that as well]

```
ip -n red  addr add 192.168.15.1 dev veth-red
```

```
ip -n blue addr add 192.168.15.2 dev veth-blue
```

- Now, both namespaces (red and blue) are having one one interface *veth-red* and *veth-blue* respectively having some IPs.
  [here IPs are 192.168.15.x range, but the Host's IP is in 192.168.1.x range]

- Now, make those interfaces up.

ip -n <namespace name> link set <interface name> up

(or)

ip netns exec <namespace name> ip link set <interface name> up

```
ip -n red  link set veth-red up
```

```
ip -n blue link set veth-blue up
```

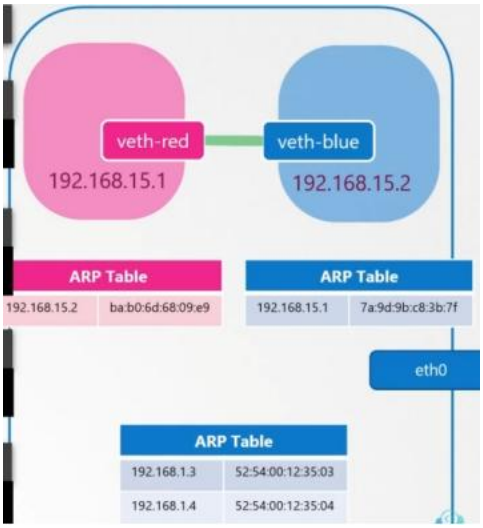
- If you try to ping *blue* namespace from *red*, it'll be able to ping.

```
ip netns exec red ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.026 ms
```

- After pinging, the arp table should be updated in both the namespaces.

```
ip netns exec red arp
Address          HWtype  HWaddress           Flags Mask            Iface
192.168.15.2     ether   ba:b0:6d:68:09:e9   C                     veth-red
```

```
ip netns exec blue arp
Address          HWtype  HWaddress           Flags Mask            Iface
192.168.15.1     ether   7a:9d:9b:c8:3b:7f   C                     veth-blue
```

- 

- If you see *host's* arp table

```
arp
Address                  HWtype  HWaddress           Flags Mask            Iface
192.168.1.3              ether   52:54:00:12:35:03   C                     eth0
192.168.1.4              ether   52:54:00:12:35:04   C                     eth0
```

  it has no idea about namespaces

- **Till now we had only 2 namespaces, now multiple namespaces**
  - You need to create a ***virtual switch*** to create a *virtual network* to which all the namespaces will be connected.
  - There are multiple option to create a virtual bridge like

    linux bridge (native solution) [we'll gonna use this]

    Open vSwitch
  - We'll create a virtual switch using

    ip link add <bridge interface name> **type bridge**

    it'll appear as a normal interface in the Host just like *eth0* interface.

```
ip link add v-net-0 type bridge
```

```
ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdis
UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> m
fq_codel state UP mode DEFAULT group default
    link/ether 02:0d:31:14:c7:a7 brd ff:ff:f
6: v-net-0: <BROADCAST,MULTICAST> mtu 1500 q
mode DEFAULT group default qlen 1000
    link/ether 06:9d:69:52:6f:61 brd ff:ff:f
```

[**v-net-0** is there as a normal interfacce]

  - Now, make the interface UP.

```
ip link set dev v-net-0 up
```

  - Previously we connected the ***veth pair*** to both the namespaces (one end to red and another to blue).

    But now, we'll connect one end to a namespace and another to *bridge (virtual switch)*.
  - First delete the previously created *veth pair* .

    ip -n <namespace name> **link delete** <veth pair end>

    (or)

    ip netns exec <namespace name> **ip link delete** <veth pair end>

[just delete one end of *veth pair*, other end will be deleted automatically]

```
ip -n red link del veth-red
```

- Now create a *veth pair* which will connect the namespace to bridge.

  (just for convention, we'll give one end *veth-red,* and another *veth-red-br*)

```
ip link add veth-red type veth peer name veth-red-br
```

similarly for *blue* namespace

```
ip link add veth-blue type veth peer name veth-blue-br
```

- Now we'll connect the *veth pair* to namespace and bridge.

  One end to namespace red:

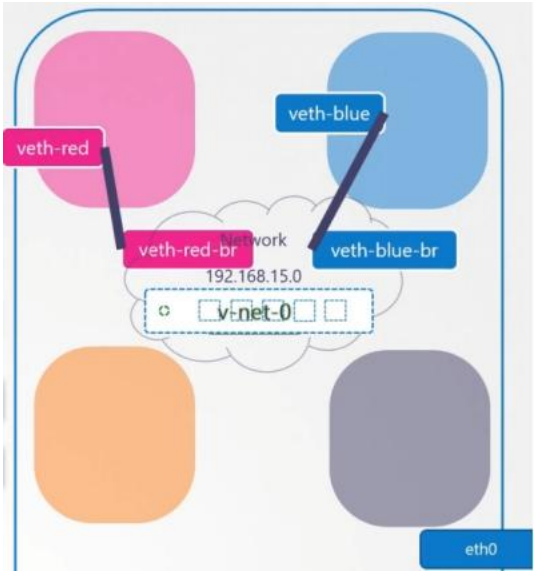
  ip link set veth-red **netns** red

  Another end to bridge:

  ip link set veth-red-br **master** v-net-o

  for *namespace,* **netns** <namespace name>, and for host **master** <interface name>

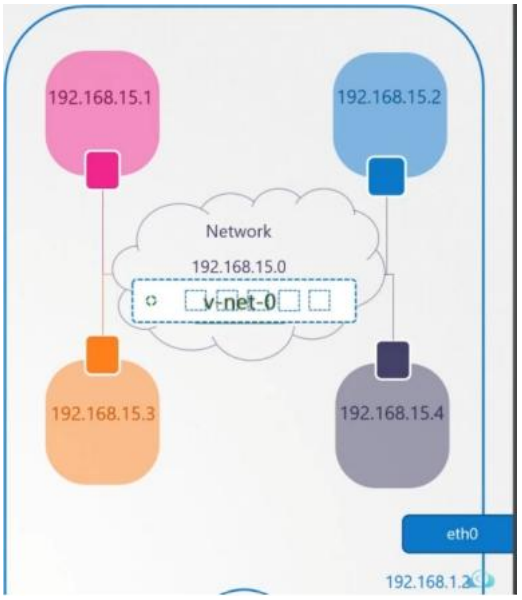
  do the same for *blue* namespace.



- Now set IP address for the link and turn them UP.

```
ip -n red addr add 192.168.15.1 dev veth-red
```

```
ip -n blue addr add 192.168.15.2 dev veth-blue
```

- ```
  ip -n red link set veth-red up
  ```
  ```
  ip -n blue link set veth-blue up
  ```
  

- But, if you now try to ping any of these namespaces from the Host, it'll be unreachable because they are in different network.

  ```
  ping 192.168.15.1
  Not Reachable!
  ```
- Now, all namespaces have some IPs but the switch doesn't have any IP. (its layer 2, ping is a layer-3 protocol, it needs an IP).
- Assign a IP ***192.168.15.5/24*** to the bridge interface (no need to manually assign subnet *192.168.15.0* as shown in the previous image).

  ip addr add 198.168.15.5/24 **dev** v-net-0

  ```
  ip addr add 192.168.15.5/24 dev v-net-0
  ```
- Now the ping will be successful.

  ```
  ping 192.168.15.1
  PING 192.168.15.1 (192.168.15.1) 56(84) bytes of data.
  64 bytes from 192.168.15.1: icmp_seq=1 ttl=64 time=0.026 ms
  ```
- Till now, only the Host and Namespaces can communicate.

  But, the namespace can't communicate with the internet.

  The only door to the outside world is the *ethernet* port on the host (**etho**).
  ```

- Lets say a device (lets say device-2) is connected to the same LAN (to which the Host is connected) and its IP is: **192.168.1.3** (host's IP is 192.168.1.2)
- Now, if we try to ping that device-2 from *blue* namespace, it'll be not reachable.

```
ip netns exec blue ping 192.168.1.3
Connect: Network is unreachable
```

If you see the ***route*** table of blue namespace, it doesn't contain any entry for *192.168.1.x* range IPs.

```
ip netns exec blue route
Destination     Gateway         Genmask         Flags Metr
192.168.15.0    0.0.0.0         255.255.255.0   U     0
```

[it just knows its own subnet]

NOTE: It is not the **default gw.** For default gw the entry looks like the below

```
Destination     Gateway         Genmask
0.0.0.0         192.168.15.1    0.0.0.0
```

- Who contains both the network interfaces i.e. bridge network interface and *etho* .

  It is ***localhost***.

  It is connected to the private network (192.168.15.x) and the outside network (192.168.1.x).
- **localhost** is the gateway that connects the 2 networks together.
- To add a gateway,

  ip route add <destination> via <gateway>
- We'll add the gateway as *192.168.15.5* (bridge interface IP) to connect to LAN (*192.168.1.0*)

  (because previously no gateway was there)

```
ip netns exec blue ip route add 192.168.1.0/24 via 192.168.15.5
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.15.0    0.0.0.0         255.255.255.0   U     0      0        0 veth-blue
192.168.1.0     192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
```

- Now if you ping *device-2* it'll no longer say not reachable.

```
ip netns exec blue ping 192.168.1.3
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
```

but, it'll not get any response back.

- Now the issue of not getting response is the below:
  - The *device-2* only recognize our host because both belong to same network (192.168.1.x).
  - When the namespace send the packet to *device-2*, it goes through the virtual switch (192.168.15.5).

    localhost acts as a gateway for connecting *bridge interface* and *ethernet* (etho).
  - So,

    namespace (192.168.15.2) ---> bridge interface (192.168.15.5) --> localhost (gateway) --> etho (192.168.1.2) --> *device-2* (192.168.1.3)
  - But, device-2 doesn't know about *192.158.15.x* ip address range.

    So it doesn't know what is the gateway or to which it should send back the packets.
  - One option is:

    you set the gateway as host IP (192.168.1.2) to reach namespaces (192.168.15.0) in ***device-2***.

    But, if you do this way, in all the devices you need to add the route like this. VERY HARD TO MANAGE.

    Then device-2 can send the response properly.
  - Another option is **NAT**

    Host rewrites source IP before packet leaves.
  - Packet before NAT

    ```
    SRC = 192.168.15.2
    DST = 192.168.1.3
    ```
  - Packet after NAT

    ```
    SRC = 192.168.1.2
    DST = 192.168.1.3
    ```
  - Now, device-2 knows this IP, and can easily send the response back.

    ```
    SRC = 192.168.1.3
    DST = 192.168.1.2
    ```
    (src is device-1, dst is host)

- Host receives the packet.
- Host NAT table remembers that this packet's destination is *192.158.15.2* , so it rewrites the destination.

  ```
  DST = 192.168.15.2
  ```
- Now, the packet is successfully sent to the namespace.

- To add **NAT**

```
iptables -t nat -A POSTROUTING -s 192.168.15.0/24 -j MASQUERADE
```

lets say packet's path is like this

```
Namespace -> Host -> Outside network
```

before packet leaves the host, linux get a last chance to edit this. This stage: **POSTROUTING** (means: after route decided, before packet exits)

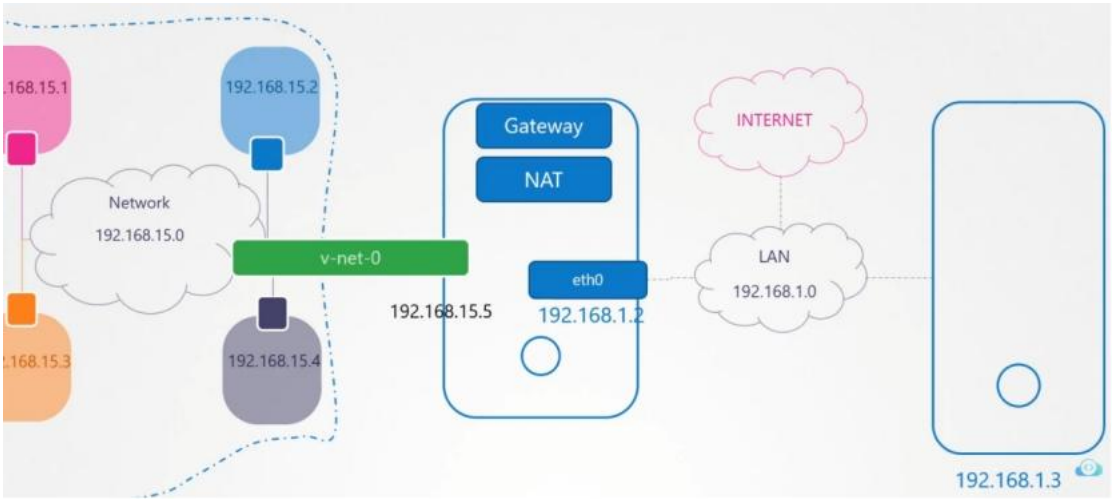
MASQUERADE means replace src IP with host IP

[192.168.15.x with 192.168.1.2]

- Now, to reach to internet, (try to ping **8.8.8.8**), you need to set the **default gateway** as bridge network interface's IP (192.168.15.2)

```
ip netns exec blue ip route add default via 192.168.15.5
```

```
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.15.0    0.0.0.0         255.255.255.0   U     0      0        0 veth-blue
192.168.1.0     192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
Default         192.168.15.5    255.255.255.0   UG    0      0        0 veth-blue
```



- **SNAT**: change source IP

  DNAT: change destination IP

- What is bridge?
  - Bridge is a ***virtual switch*** in Linux.

    Similar to *physical switches, WiFi route switches*.
  - Linux can create software switches.

    ip link add <switch name> **type bridge**

    you can attach interfaces to it [just like plugging cables into switch ports]
  - Bridge is required for pods of the *same node* to communicate.
  - Now all pods are on same virtual LAN (vlan).
  - NOTE: if you create a bridge in linux, it'll be inside HOST namespace.
- What is **veth pair**?
  - Pods are inside a separate network namespace than the host.

    Need something to connect those.
  - One end of **veth** is connected to Host Network Namespace,

    Another end is connected to Pod Network Namespace.
  - 

    - **veth-pod** (veth's side connected to pod network namespace) will be renamed **etho** inside Pod network namespace.
- What CNI actually does?
  - Create a network namespace (Pod network).
  - Create a **veth** pair (virtual cable).
  - Connects one end to bridge, another to pod namespace.
  - Give Pod an IP.
  - Add **route + gateway**
  - Enable NAT (optional for internet).

IPv4 & IPv6 Networking & Hostname Resolution

- **ip** command just shows and configure the current network settings, but this is not permanent.
  - Different system uses different tools for these.
  - Ubuntu uses **NetPlan**.
  - **netplan get**

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      match:
        macaddress: "02:aa:f8:fc:c7:1b"
      dhcp4: true
      dhcp6: true
      set-name: "enp0s3"
    enp0s8:
      addresses:
      - "192.168.56.21/24"
```

    for enp0s3 : dhcp4 & dhcp6 are true.

    For enp0s8: static IP is assigned.

  - ***netplan*** config files are present inside the directory **/etc/netplan/**

```
root@node01:/etc/netplan# ls
50-cloud-init.yaml  50-vagrant.yaml
```

  - We can create a config file by ourselves.

```
root@vm1:/etc/netplan# cat 99-mysettings.yaml
network:
    ethernets:
        enp0s8:
            dhcp4: false
            dhcp6: false
            addresses:
              - 10.0.0.9/24
              - fe80::921b:eff:fe3d:abcd/64
    version: 2
```

    [name of the file does matter; all the files will be selected in alphabetical order]

  - You can apply the configs with

    netplan apply

    but, after applying the connection might be lost. So use

    netplan try

    it'll rollback in case of any issue.

```
enp0s8: <BROADCAST,MULTICAST
 link/ether 08:00:27:c9:fd:5
 inet 10.10.10.10/24 brd 10.
    valid_lft forever prefer
 inet 10.0.0.9/24 brd 10.0.0
    valid_lft forever prefer
 inet6 fe80::921b:eff:fe3d:a
```

- Now if we execute **netplan get** command, it'll display the new configs

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      match:
        macaddress: "02:aa:f8:fc:c7:1b"
      dhcp4: true
      dhcp6: true
      set-name: "enp0s3"
    enp0s8:
      addresses:
      - "10.10.10.10/24"
      - "10.0.0.9/24"
      - "fe80::921b:eff:fe3d:abcd/64"
      dhcp4: false
      dhcp6: false
```

- Now, even after rebooting, these changes will be kept applied. It is permanent.
- Some more fields can be added, like *nameservers*, *gateway*

```
network:
    ethernets:
        enp0s8:
            dhcp4: false
            dhcp6: false
            addresses:
              - 10.0.0.9/24
              - fe80::921b:eff:fe3d:abcd/64
            nameservers:
              addresses:
                - 8.8.4.4
                - 8.8.8.8
            routes:
              - to: 192.168.0.0/24
                via: 10.0.0.100
              - to: default
                via: 10.0.0.1
    version: 2
```

  - if you add gateway using ip route add command, no need to give network interface. Kernel itself finds out the interface seeing the gateway IP.

- But if gateway is not there and you don’t want to add it via ip route add every time, then just write in required network interface level (which would have been selected by kernel by default if route added via command ip route add)]

==========

Start, Stop & Check status of Network Interfaces

- **ss -ltunp**
  - **-l** : listening
  - **-t** : TCP connections
  - **-u** : UDP connections
  - **-n** : numeric values
  - **-p** : processes

- Linux Networking Commands
  - **ip addr**

    displays all the network interfaces

    ip -c addr

    display with colors, readable. [~~ip addr -c~~]
  - **ip link set dev <**network interface**> up**

    make the net interface up. [for making down, just replace *up* with *down*]
  - **ip addr add <**IP address**>/<**subnet length**> dev <**network interface**>**

    assign IP to network interface.

    Replace **add** with **delete** to delete the IP on a network interface.

-